FINCOMERCIAL, S.A.

Analítica inteligente para anticipar el riesgo hipotecario

VALIDACIÓN Y PRUEBAS · FASE II

FinPredict

Informe técnico de validación y pruebas

PROPÓSITO DEL DOCUMENTO

Trazabilidad de las pruebas del modelo, integración FastAPI–Streamlit, contenedores Docker y evidencias visuales integradas por sección.

CAMPO	INFORMACIÓN
Organización	FinComercial, S.A.
Responsable	Lizbeth Bruno Lorenzana
Puesto	AI Lead
Asignatura	Gestión de Proyectos de Inteligencia Artificial
Solución	FinPredict – Fase II
Ámbito	Prototipo académico · Datos sintéticos
Versión documental	2.1 | 7 de noviembre de 2026
Estado	Pruebas locales, API, frontend y Docker documentados

Documento preparado para consulta, evaluación académica y trazabilidad técnica en el repositorio del proyecto.

1. Objetivo

Verificar la correcta ejecución del modelo predictivo de incumplimiento hipotecario; su consistencia con los resultados obtenidos en la Actividad 6; la validación de entradas y solicitudes HTTP; la integración funcional entre frontend y backend; y la ejecución reproducible de los servicios mediante contenedores Docker. El propósito de estas pruebas es aportar evidencia técnica de **funcionamiento, integración y portabilidad local** de FinPredict. No se busca demostrar aptitud productiva ni validar decisiones reales de otorgamiento de crédito.

2. Alcance y componentes

Las pruebas se realizaron con un modelo de **regresión logística** previamente entrenado con datos sintéticos. El artefacto serializado en formato Joblib contiene el pipeline de inferencia, las **15 variables predictoras** y el umbral operativo de clasificación de **0.75 (75%)**. **Componentes evaluados:**

- `model/finpredict_modelo.joblib`: artefacto del modelo entrenado.
- `scripts/validar_modelo.py`: comparación con predicciones históricas.
- `scripts/validar_casos_extremos.py`: escenarios financieros seleccionados.
- `scripts/validar_entradas.py`: reglas preliminares de validación.
- `api/main.py`: servicio de inferencia con FastAPI.
- `api/schemas.py`: contrato de entrada mediante Pydantic.
- `tests/test_api.py`: pruebas automatizadas de endpoints.
- `frontend/app.py`: interfaz Streamlit.
- `requirements.txt`: dependencias de Python utilizadas.
- `Dockerfile.api` y `Dockerfile.frontend`: construcción de imágenes.
- `.dockerignore`: exclusión de archivos innecesarios del contexto de construcción.
- `compose.yaml`: definición y comunicación de los servicios contenerizados.

Entornos probados: ejecución local en Windows con entorno virtual de Python y ejecución contenerizada mediante Docker Desktop (contenedores Linux).

3. Prueba 01: Consistencia histórica

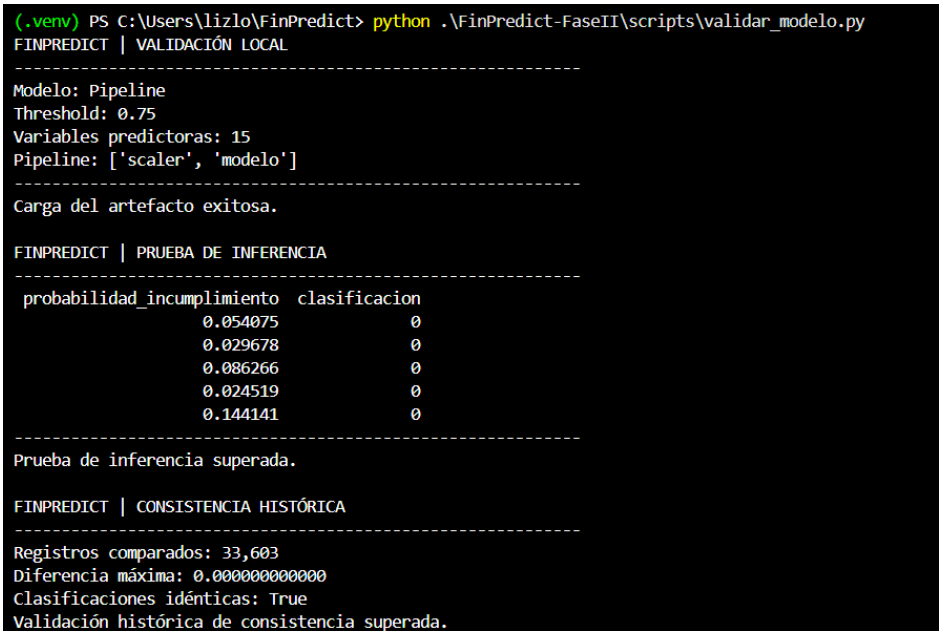
Script: `scripts/validar_modelo.py` **Objetivo:** Verificar que el modelo exportado reproduce los resultados históricos del conjunto de evaluación de la Actividad 6.

Resultados

Indicador	Resultado
Registros comparados	33,603
Diferencia máxima reportada	0.000000000000
Coincidencia de clasificaciones	100%

Indicador	Resultado
Estado	SATISFACTORIO

Evidencia: `docs/evidencias/01_consistencia_historica.png` **Conclusión:** El artefacto exportado reprodujo las predicciones históricas dentro de la tolerancia numérica aplicada. Esta comprobación respalda la consistencia del modelo serializado frente a los resultados de referencia, pero no constituye una nueva evaluación de desempeño predictivo.



Evidencia visual · 01_consistencia_historica.png · Consistencia histórica del modelo exportado.

4. Prueba 02: Casos extremos

Script: `scripts/validar_casos_extremos.py` **Objetivo:** Comprobar la ejecución de inferencias sobre registros seleccionados con características financieras extremas.

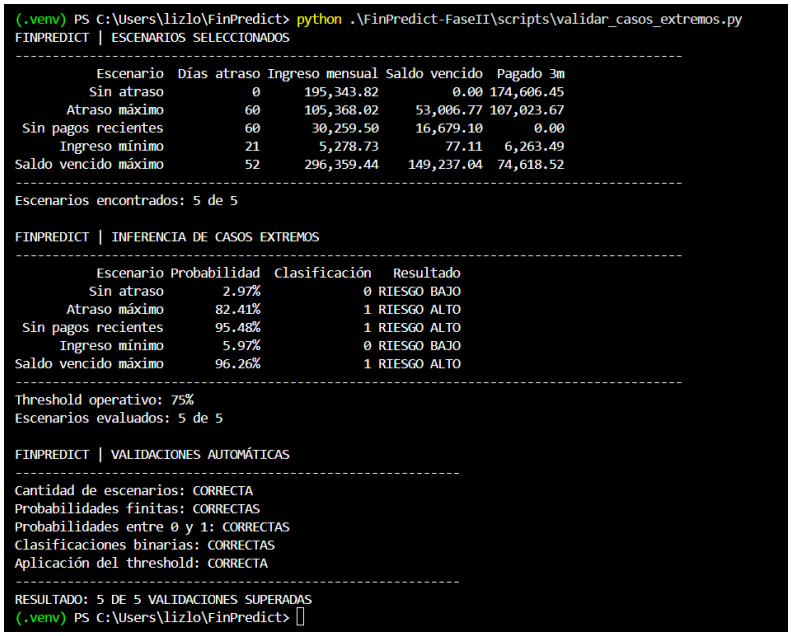
Resultados

Escenario	Probabilidad	Clasificación
Sin atraso	2.97%	Riesgo bajo
Atraso máximo	82.41%	Riesgo alto
Sin pagos recientes	95.48%	Riesgo alto
Ingreso mínimo	5.97%	Riesgo bajo
Saldo vencido máximo	96.26%	Riesgo alto

Origen de datos: Cuatro escenarios del conjunto de evaluación y un escenario del conjunto de desarrollo (sin pagos recientes). **Verificaciones automáticas:**

- Cantidad de escenarios: correcta.
- Probabilidades finitas: correcto.
- Probabilidades entre 0 y 1: correcto.
- Clasificaciones binarias: correcto.
- Aplicación del umbral operativo: correcta.

Resultado: 5 de 5 verificaciones superadas. **Evidencia:** `docs/evidencias/02_validacion_casos_extremos.png`
Conclusión: El modelo ejecutó inferencias sobre los cinco registros seleccionados y produjo salidas técnicamente válidas. El ejercicio no constituye una evaluación exhaustiva de todos los extremos posibles ni un análisis de robustez estadística de la población.



Evidencia visual · 02_validacion_casos_extremos.png · Inferencia sobre cinco casos extremos seleccionados.

5. Prueba 03: Validación de entradas

Script: `scripts/validar_entradas.py` **Objetivo:** Verificar que las funciones de validación identifican y rechazan entradas incompletas, valores numéricos inválidos y datos que incumplen restricciones financieras preliminares.

Metodología

Se desarrollaron tres grupos de pruebas mediante aserciones de Python y manejo de excepciones. Los datos de prueba se diseñaron para verificar reglas específicas y no representan solicitudes reales de crédito.

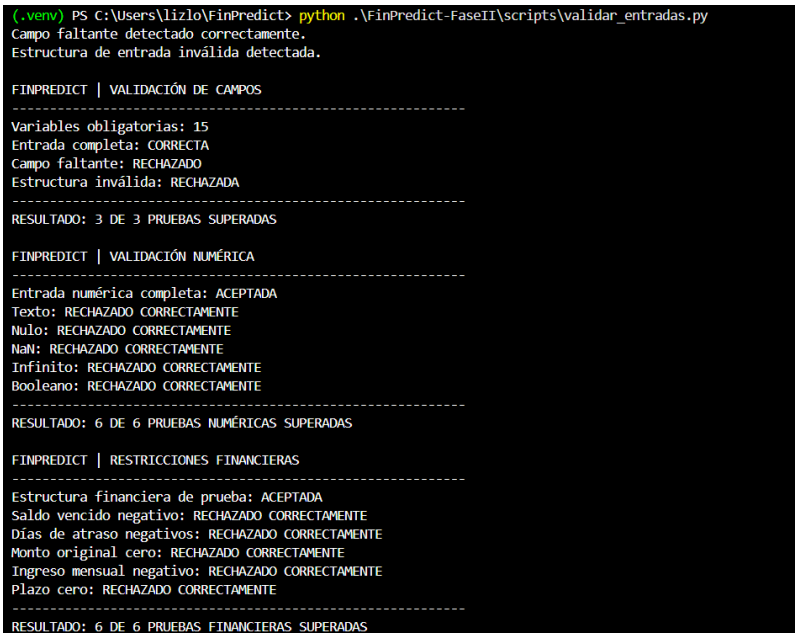
Resultados

Grupo de validación	Pruebas	Resultado
Campos obligatorios	3	Superadas
Tipos y valores numéricos	6	Superadas
Restricciones financieras	6	Superadas
Total	**15**	**15 superadas**

Casos verificados:

- Presencia de las 15 variables predictoras.
- Rechazo de campos obligatorios faltantes y estructuras incorrectas.
- Rechazo de texto, valores nulos, NaN, infinito y booleanos, según las reglas implementadas.
- Rechazo de valores negativos en variables definidas como no negativas.
- Rechazo de valores cero o negativos en variables definidas como estrictamente positivas.

Evidencia: `docs/evidencias/03_validacion_entradas.png` **Conclusión:** Las funciones superaron las 15 pruebas ejecutadas. **Estas reglas sí fueron integradas posteriormente en FastAPI**, junto con las validaciones de estructura y tipos de Pydantic, como se demuestra en las secciones 6 y 7. Las restricciones financieras siguen siendo preliminares y requieren aprobación del área de negocio para cualquier aplicación real.



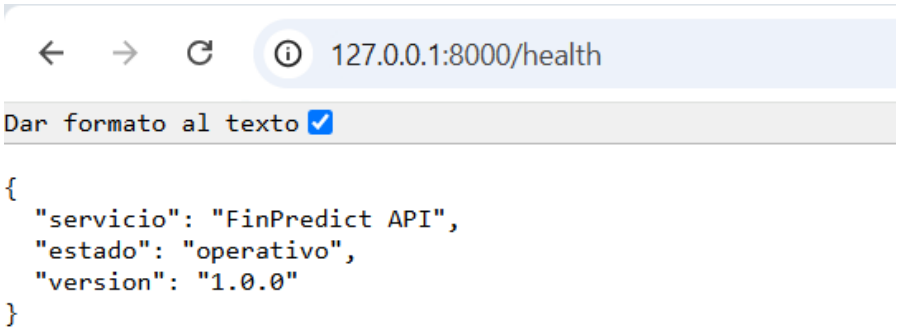
Evidencia visual · 03_validacion_entradas.png · Resultados de validación de entradas locales.

6. Prueba 04: Integración y validación local de la API

Componentes: `api/main.py`, `api/schemas.py`, `scripts/validar_entradas.py`, `tests/test_api.py`, `frontend/app.py`. **Objetivo:** Verificar la inferencia mediante una API REST, el rechazo de solicitudes inválidas y la comunicación con la interfaz Streamlit.

6.1. Disponibilidad del servicio

Se ejecutó `GET /health` desde Swagger UI sobre el backend local. **Resultado:** HTTP 200, servicio operativo, versión 1.0.0. **Evidencia:** `docs/evidencias/04_api_health.png`



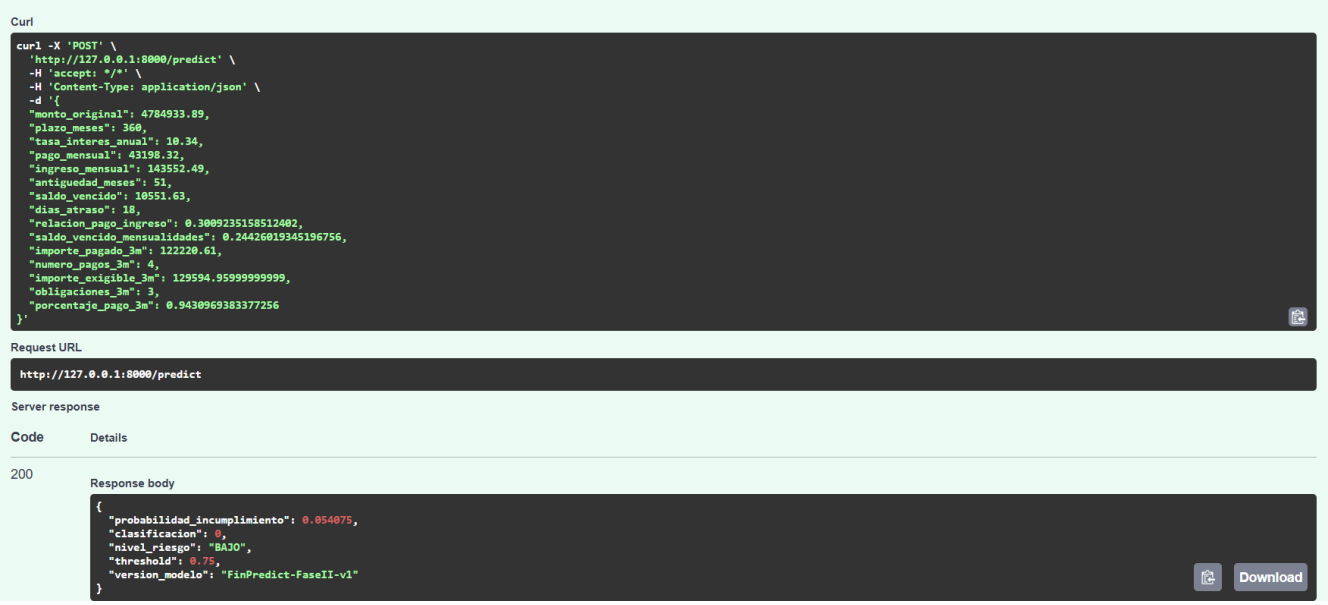
Evidencia visual · 04_api_health.png · Disponibilidad del servicio local mediante GET /health.

6.2. Inferencia mediante HTTP

Se ejecutó `POST /predict` con un registro sintético de evaluación.

Indicador	Resultado
Respuesta HTTP	200
Probabilidad de incumplimiento	5.4075%
Clasificación	0 — riesgo bajo
Umbral operativo	75%

La probabilidad devuelta por la API, expresada en escala de 0 a 1 y redondeada a seis decimales (`0.054075`), corresponde al resultado local de referencia. **Evidencia:** `docs/evidencias/05_api_primera_prediccion.png`



Evidencia visual · 05_api_primera_prediccion.png · Primera inferencia local en Swagger UI.

6.3. Rechazo de solicitudes inválidas

Se enviaron solicitudes deliberadamente incorrectas para verificar el comportamiento del contrato de entrada y de las reglas financieras.

Escenario	Código HTTP	Resultado
Campos financieros en cero	422	Rechazado
Ingreso mensual negativo	422	Rechazado
Campo `dias_atraso` faltante	422	Rechazado
Texto en `dias_atraso`	422	Rechazado

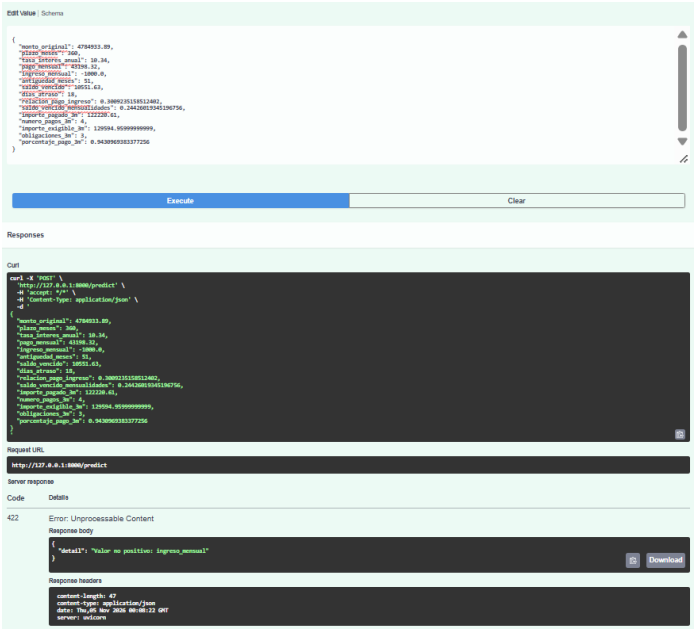
Evidencias:

- `docs/evidencias/06_api_validacion_financiera.png`



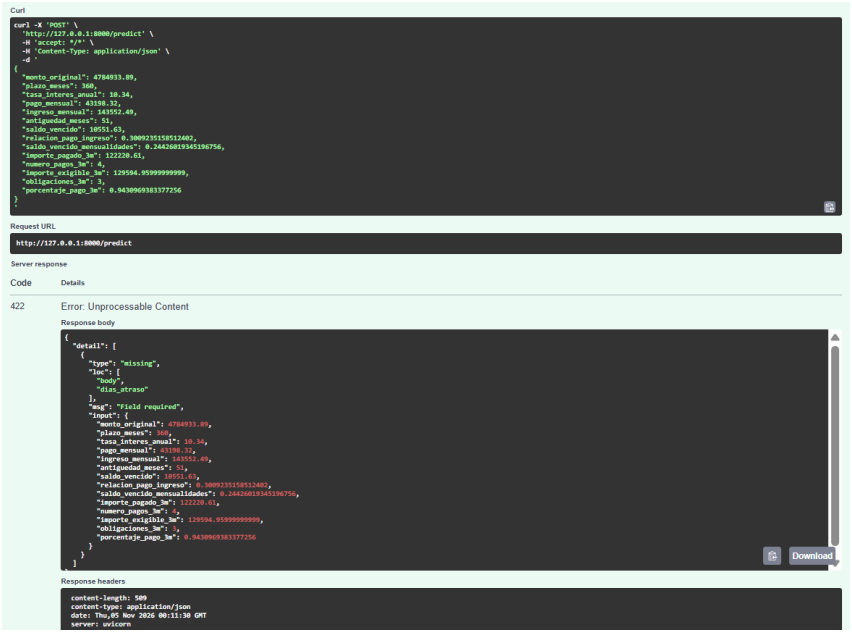
Evidencia visual · 06_api_validacion_financiera.png · Rechazo local por restricciones financieras.

- docs/evidencias/07_api_ingreso_negativo.png



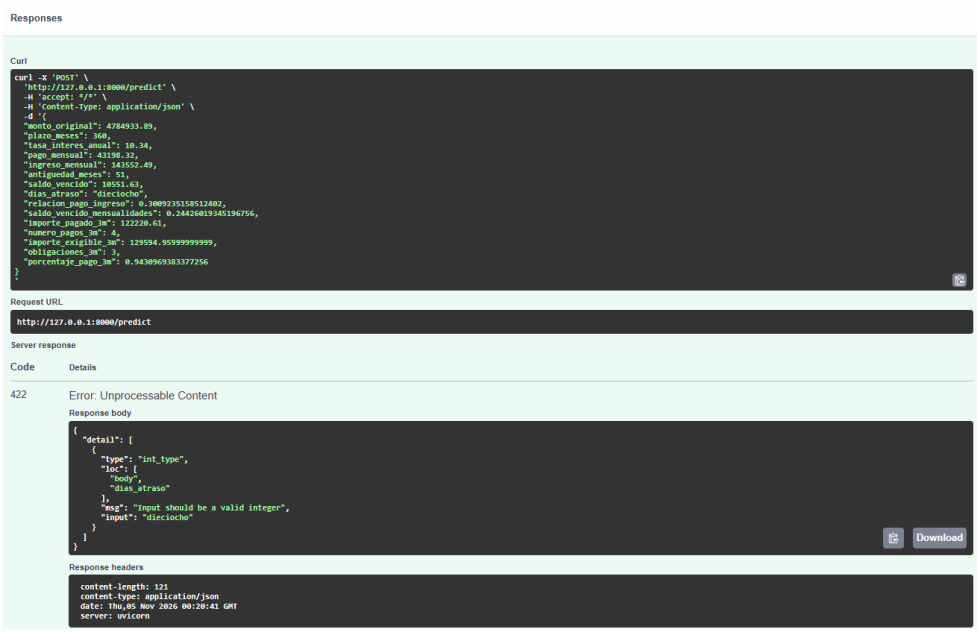
Evidencia visual · 07_api_ingreso_negativo.png · Rechazo local por ingreso mensual negativo.

- docs/evidencias/08_api_campo_obligatorio.png



Evidencia visual · 08_api_campo_obligatorio.png · Rechazo local por campo obligatorio faltante.

- `docs/evidencias/09_api_tipo_incorrecto.png`



Evidencia visual · 09_api_tipo_incorrecto.png · Rechazo local por tipo de dato incorrecto.

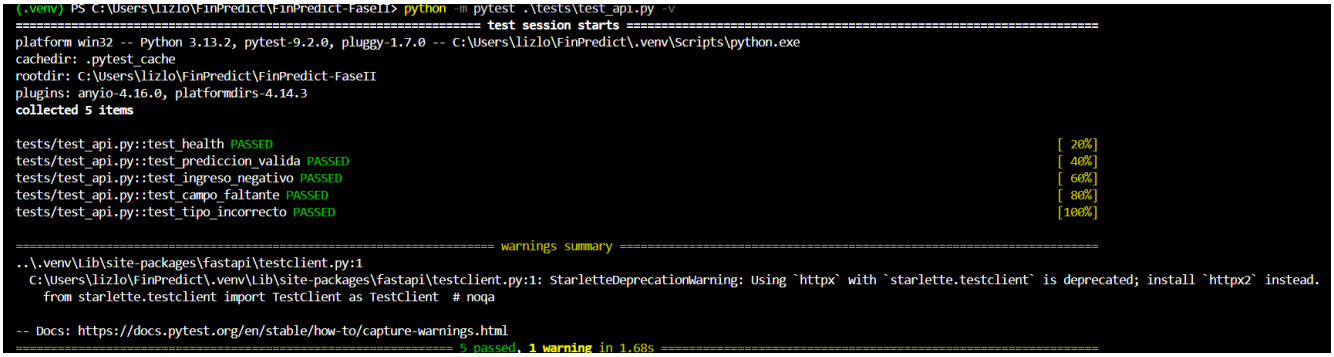
6.4. Pruebas HTTP automatizadas

Herramientas: Pytest, HTTPX y `TestClient`.

Prueba	Resultado
`test_health`	PASSED
`test_prediccion_valida`	PASSED
`test_ingreso_negativo`	PASSED

Prueba	Resultado
test_campo_faltante	PASSED
test_tipo_incorrecto	PASSED

Resultado: 5 pruebas superadas, 0 fallidas y 1 advertencia de compatibilidad futura de dependencias. La advertencia no impidió finalizar las pruebas, aunque debe revisarse antes de actualizar bibliotecas. **Evidencia:** docs/evidencias/10_api_pruebas_automatizadas.png



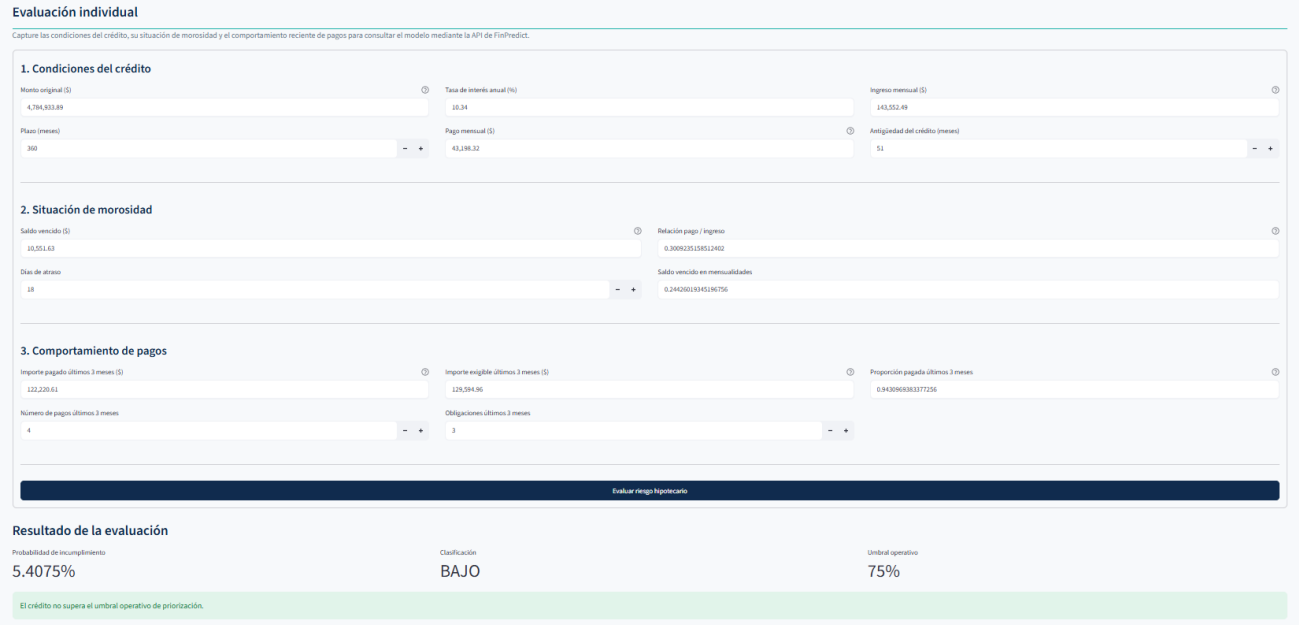
Evidencia visual · 10_api_pruebas_automatizadas.png · Pruebas automatizadas de la API con Pytest.

6.5. Integración frontend/backend con Streamlit

Objetivo: Verificar que la interfaz captura las 15 variables, envía una solicitud HTTP a FastAPI y presenta la predicción al usuario. **Metodología:** Se utilizó el mismo registro sintético previamente validado mediante inferencia local y Swagger.

Indicador	Resultado
Probabilidad de incumplimiento	5.4075%
Clasificación	Riesgo bajo
Umbral operativo	75%
Comunicación frontend/backend	Satisfactoria

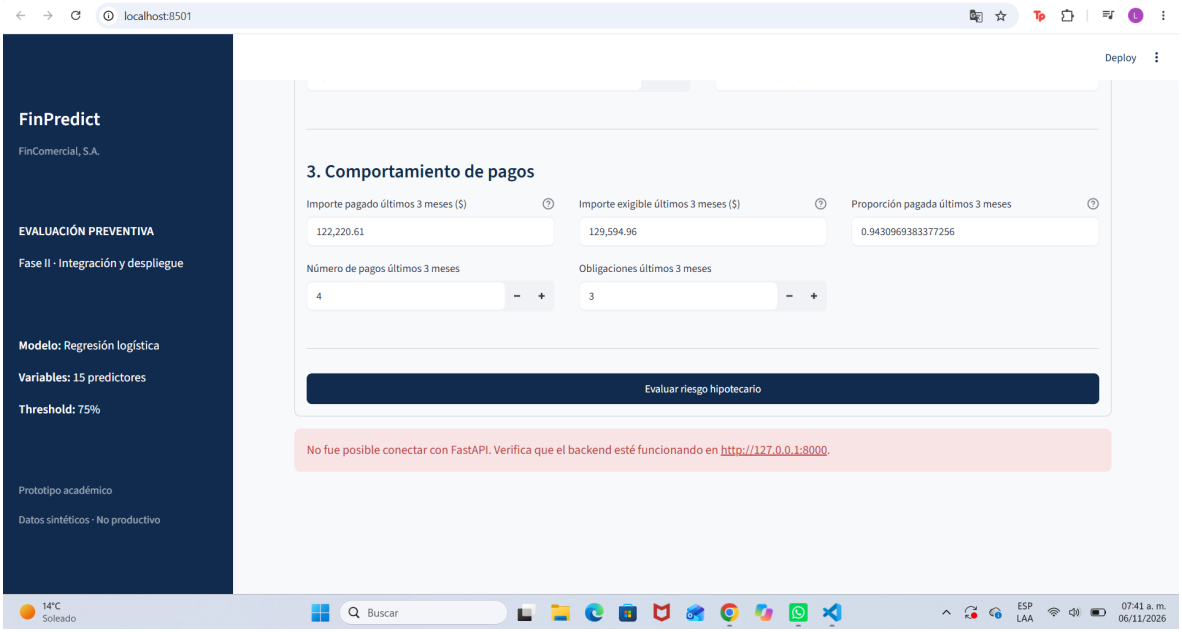
Evidencia: docs/evidencias/11_integracion_streamlit_fastapi.png



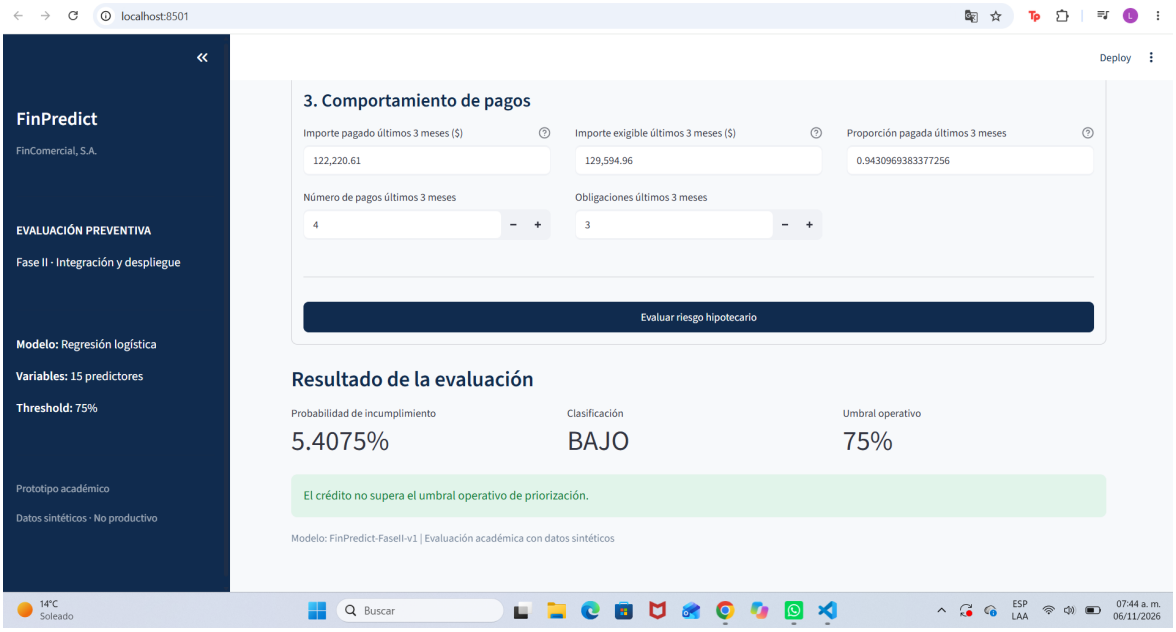
Evidencia visual · 11_integracion_streamlit_fastapi.png · Integración funcional entre Streamlit y FastAPI.

6.6. Desconexión y recuperación de la API local

Objetivo: Evaluar la respuesta de Streamlit ante la interrupción temporal del backend. **Prueba 1 — API desconectada:** Se detuvo manualmente FastAPI y se solicitó una evaluación desde Streamlit. La interfaz mostró un error controlado sin cerrarse. **Evidencia:** `docs/evidencias/12_frontend_api_desconectada.png` **Prueba 2 — Recuperación:** Se reinició FastAPI sin reiniciar Streamlit y se evaluó nuevamente el mismo registro. Se restableció la comunicación y se recuperó la probabilidad de **5.4075%**, la clasificación de riesgo bajo y el umbral de **75%**. **Evidencia:** `docs/evidencias/13_frontend_recuperacion_api.png` **Conclusión:** La interfaz manejó la desconexión mediante un mensaje controlado y pudo volver a consumir la API tras su restablecimiento manual. Esto **no demuestra** reintentos automáticos, alta disponibilidad ni tolerancia a fallas entre réplicas.



Evidencia visual · 12_frontend_api_desconectada.png · Manejo controlado cuando la API local no está disponible.



Evidencia visual · 13_frontend_recuperacion_api.png · Recuperación del frontend tras restablecer la API.

6.7. Conclusión de las pruebas locales

La API ejecutó inferencias válidas, rechazó las solicitudes incorrectas ensayadas y superó las cinco pruebas HTTP automatizadas. La integración con Streamlit y la recuperación funcional después de una interrupción manual también fueron satisfactorias. La contenerización y su evaluación específica se describen por separado en la sección 7.

7. Prueba 05: Contenerización y despliegue local con Docker

Objetivo: Construir y ejecutar de forma separada el backend y el frontend de FinPredict en contenedores Linux, validar su comunicación y comprobar el comportamiento de inferencia y validación de solicitudes desde el despliegue contenerizado.

7.1. Componentes de la solución

Componente	Archivo o servicio	Función
Imagen de API	`Dockerfile.api` / `finpredict-api:v1`	Ejecutar FastAPI y cargar el modelo Joblib
Imagen de frontend	`Dockerfile.frontend` / `finpredict-frontend:v1`	Ejecutar la interfaz Streamlit
Dependencias	`requirements.txt`	Instalar bibliotecas utilizadas en la ejecución
Exclusiones	`.dockerignore`	Evitar el envío de archivos innecesarios al contexto de construcción
Coordinación	`compose.yaml`	Definir servicios, puertos y red interna

Componente	Archivo o servicio	Función
Contrato de conexión	`FINPREDICT_API_URL`	Configurar el destino del backend sin modificar el código fuente

Se utilizó la imagen base `python:3.13-slim`. Ambas imágenes incluyen únicamente los componentes copiados por sus respectivos Dockerfiles, no la totalidad del repositorio.

7.2. Construcción de la imagen del backend

Comando utilizado:

```
docker build -f Dockerfile.api -t finpredict-api:v1 .
```

Resultado observado: Construcción finalizada correctamente (`12/12 FINISHED`). La imagen incorpora el servicio FastAPI, el modelo serializado y el módulo de validación financiera necesario para la inferencia. **Evidencia:** `docs/evidencias/14_docker_build_api.png`



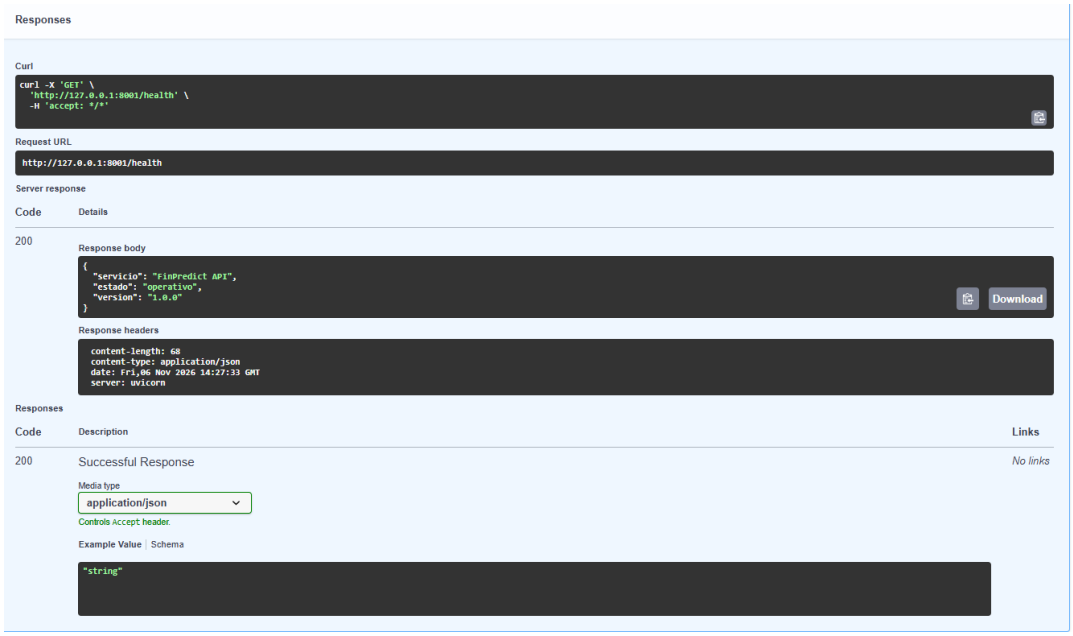
Evidencia visual · 14_docker_build_api.png · Construcción de la imagen Docker del backend.

7.3. Ejecución independiente de la API en Docker

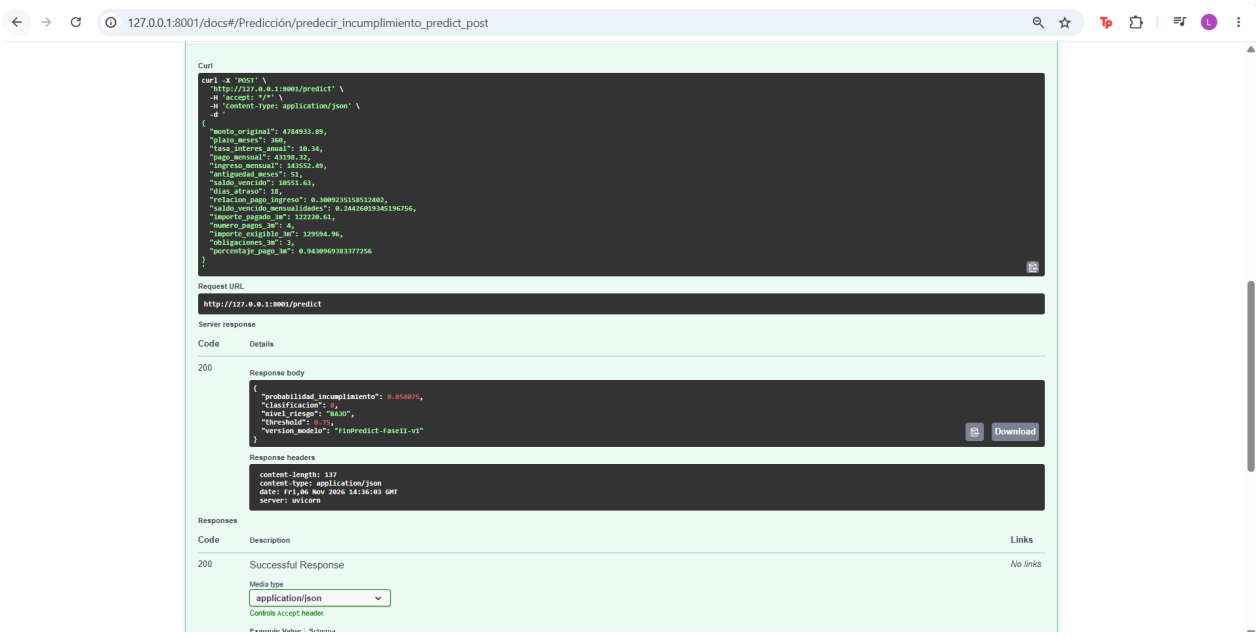
Se inició el backend como contenedor independiente, publicando únicamente en la interfaz local de Windows el puerto `8001` hacia el puerto `8000` del contenedor. **Comando utilizado:**

```
docker run --name finpredict-api-test -p 127.0.0.1:8001:8000 finpredict-api:v1
```

Disponibilidad: La consulta `GET http://127.0.0.1:8001/health` devolvió HTTP **200**, con estado `operativo` y versión `1.0.0`. **Evidencia:** `docs/evidencias/15_docker_api_health.png` **Inferencia:** La solicitud `POST http://127.0.0.1:8001/predict` con el crédito sintético de referencia devolvió HTTP **200**, probabilidad `0.054075`, clasificación `0` (riesgo bajo) y umbral `0.75`. **Evidencia:** `docs/evidencias/16_docker_api_prediccion.png` **Conclusión:** Para el registro utilizado, la API ejecutada dentro del contenedor Linux reprodujo el resultado obtenido previamente en el entorno local de Windows. Se verificó **consistencia de inferencia en un caso de referencia**, no equivalencia exhaustiva de todos los posibles resultados.



Evidencia visual · 15_docker_api_health.png · Verificación del health check de la API en contenedor.



Evidencia visual · 16_docker_api_prediccion.png · Predicción HTTP desde la API ejecutada en contenedor.

7.4. Construcción de la imagen del frontend

Comando utilizado:

```
docker build -f Dockerfile.frontend -t finpredict-frontend:v1 .
```

Resultado observado: Construcción finalizada correctamente (`10/10 FINISHED`). Docker reutilizó capas previamente almacenadas en caché, reduciendo el tiempo de esta construcción. **Evidencia:** `docs/evidencias/17_docker_build_frontend.png`

```
(.venv) PS C:\Users\lizlo\FinPredict\FinPredict-FaseII>
>> docker build -f Dockerfile.frontend -t finpredict-frontend:v1 .
>>
[+] Building 1.3s (10/10) FINISHED
 => [internal] load build definition from Dockerfile.frontend
 => => transferring dockerfile: 740B
 => [internal] load metadata for docker.io/library/python:3.13-slim
 => [internal] load .dockerignore
 => => transferring context: 606B
 => [1/5] FROM docker.io/library/python:3.13-slim@sha256:09a17cad02668ae61ece136b4fd36ac3849e14f670ebedb637ea3e15580525bc
 => => resolve docker.io/library/python:3.13-slim@sha256:09a17cad02668ae61ece136b4fd36ac3849e14f670ebedb637ea3e15580525bc
 => [internal] load build context
 => => transferring context: 17.84kB
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY frontend/ ./frontend/
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:e633dbcf7982968bb3165aafd4d5aa3893c68316bf93677f21b2dcd70fbbcc5c
 => => exporting config sha256:0dade5dd4009afea0a946e72f163abfd11682431116ddd60fd053efbb14952ee
 => => exporting attestation manifest sha256:be057b6a1a927f84bd14388e56f1132779ad500c72eecbf15fec9d3f1927d6a8
 => => exporting manifest list sha256:b4721a5d63fc56a542b00ecfaa18be2ad98c250149245f6cf8ef6bd7eb5ed576
 => => naming to docker.io/library/finpredict-frontend:v1
 => => unpacking to docker.io/library/finpredict-frontend:v1
```

Evidencia visual · 17_docker_build_frontend.png · Construcción de la imagen Docker del frontend.

7.5. Integración de servicios mediante Docker Compose

Archivo de configuración: `compose.yaml` **Comandos utilizados:**

```
docker compose config --quiet

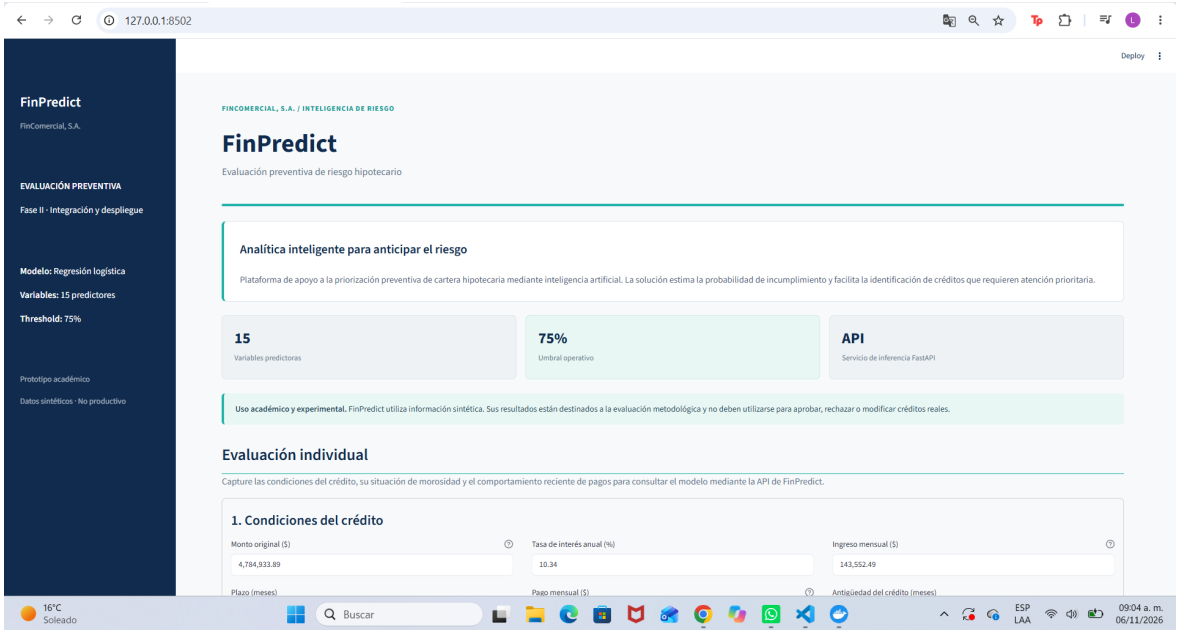
docker compose up -d
```

El primer comando validó la configuración sin reportar errores. El segundo creó la red interna e inició los servicios `api` y `frontend`. **Puertos del despliegue local con Compose:**

Servicio	Dirección accesible desde Windows	Puerto interno
Streamlit	`http://127.0.0.1:8502`	8501
FastAPI	`http://127.0.0.1:8002`	8000
Swagger UI	`http://127.0.0.1:8002/docs`	8000

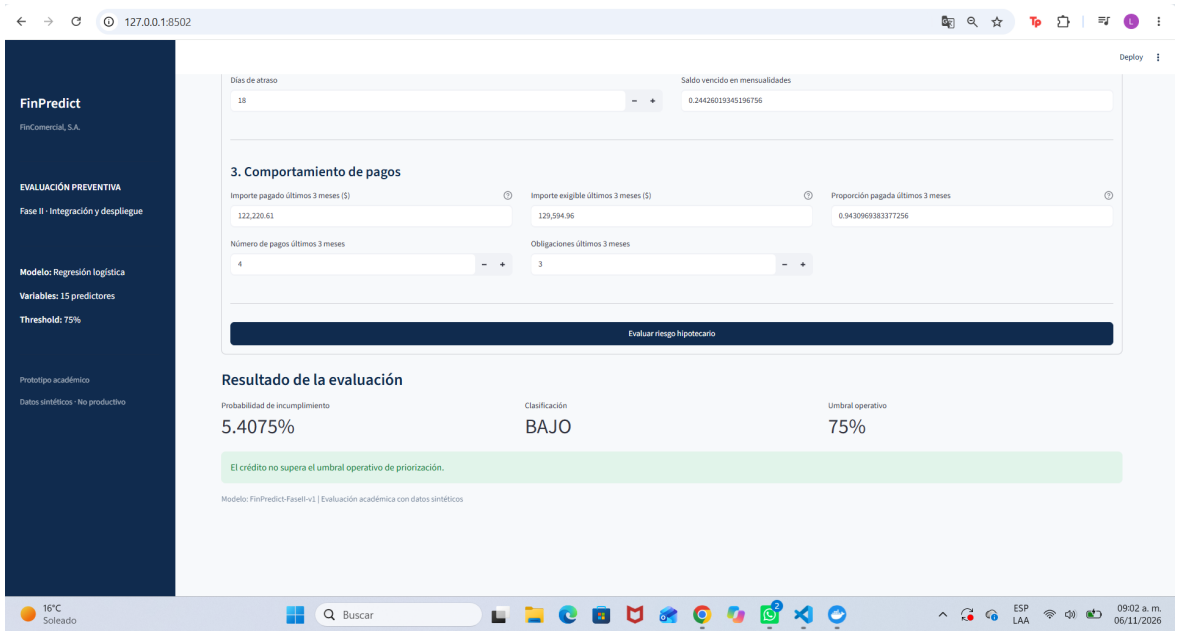
Comunicación interna: Streamlit utiliza la variable `FINPREDICT_API_URL=http://api:8000/predict`. El nombre `api` identifica al backend dentro de la red de Docker Compose; `127.0.0.1` dentro del frontend no apuntaría al otro contenedor. **Resultado end-to-end:** La interfaz del puerto `8502` mostró correctamente la aplicación y, al evaluar el registro sintético de referencia, obtuvo **5.4075%**, clasificación **BAJO** y umbral **75%**. **Evidencias:**

- `docs/evidencias/18_docker_compose_frontend.png`



Evidencia visual · 18_docker_compose_frontend.png · Ejecución del frontend mediante Docker Compose.

- `docs/evidencias/19_docker_compose_prediccion.png`



Evidencia visual · 19_docker_compose_prediccion.png · Predicción end-to-end con Docker Compose.

Conclusión: Se comprobó el flujo navegador → Streamlit contenerizado → FastAPI contenerizado → modelo → respuesta a la interfaz. Este despliegue es **local y académico**, no un despliegue público en la nube.

7.6. Reinicio y estado de servicios

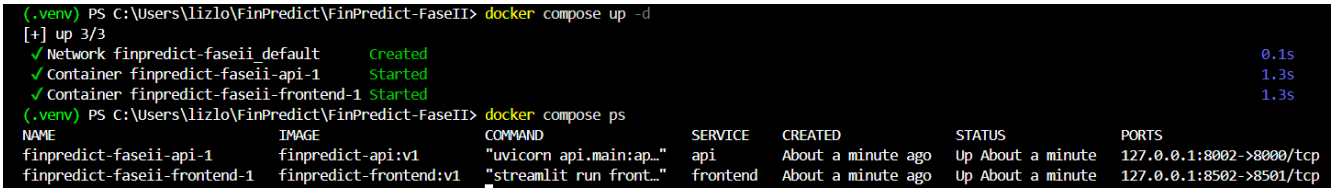
Después de detener y retirar los contenedores de Compose, se comprobó que no había servicios creados mediante `docker compose ps -a`. Posteriormente, se ejecutó de nuevo `docker compose up -d`, sin repetir la construcción de imágenes. **Comando de verificación:**

```
docker compose ps
```

Resultado observado: Los dos servicios aparecieron en estado `Up`, con las imágenes y puertos esperados.

Servicio	Imagen	Puerto publicado
api	finpredict-api:v1	127.0.0.1:8002->8000/tcp
frontend	finpredict-frontend:v1	127.0.0.1:8502->8501/tcp

Evidencia: docs/evidencias/20_docker_compose_servicios_activos.png **Alcance de la verificación:** Se comprobó el reinicio y la permanencia en estado Up durante la observación; no se realizó una prueba de carga o estabilidad prolongada.



Evidencia visual · 20_docker_compose_servicios_activos.png · Servicios y puertos activos en Docker Compose.

7.7. Revisión de registros y disponibilidad del backend

Comando utilizado:

```
docker compose logs --tail=30
```

Resultados observados: FastAPI registró Application startup complete y el inicio de Uvicorn en 0.0.0.0:8000. Streamlit informó disponibilidad en 0.0.0.0:8501. Las líneas revisadas no mostraron errores de arranque. El aviso de detección de WSL y utilización de vigilancia de archivos por sondeo fue informativo; no impidió ejecutar la aplicación. Se comprobó además el backend de Compose mediante:

```
Invoke-RestMethod -Uri "http://127.0.0.1:8002/health" -Method Get
```

Resultado: servicio FinPredict API, estado operativo, versión 1.0.0. **Nota de exposición:** Las URLs externas que puedan aparecer en los logs de Streamlit no acreditan publicación pública. En la configuración probada, los puertos de Compose se vincularon a 127.0.0.1 en Windows.

7.8. Validación de solicitudes inválidas en Docker Compose

Se ejecutaron las siguientes pruebas contra POST http://127.0.0.1:8002/predict, utilizando Swagger UI.

Prueba	Dato enviado	HTTP	Resultado observado	Evidencia
Restricción financiera	ingreso_mensual = -1000.0	422	Valor no positivo: ingreso_mensual	21_docker_validacion_ingreso_negativo.png
Campo obligatorio	Se omitió dias_atraso	422	type: missing; Field required	22_docker_campo_obligatorio.png
Tipo incompatible	dias_atraso = "dieciocho"	422	type: int_type; se requiere un entero	23_docker_validacion_tipo_incorrecto.png

Las tres solicitudes fueron rechazadas antes de ejecutar una inferencia válida. Los archivos de evidencia se encuentran en docs/evidencias/. **Conclusión:** Los controles de validación financiera, presencia de campos y tipos de datos continuaron funcionando en el despliegue contenerizado.

7.9. Reconstrucción y prueba de regresión

Objetivo: Verificar que FinPredict puede reconstruir sus imágenes desde los Dockerfiles y conservar el comportamiento predictivo del caso de referencia. **Procedimiento:**

1. Se incorporaron instrucciones `build` en `compose.yaml`.
2. Se validó la configuración con `docker compose config --quiet`.
3. Se ejecutó `docker compose up -d --build`.
4. Se confirmó la construcción de las imágenes del backend y frontend, aprovechando las capas de caché disponibles.
5. Se evaluó nuevamente el crédito sintético de referencia desde Streamlit en el puerto 8502.

Resultados:

Indicador	Resultado
Imagen FastAPI	Built
Imagen Streamlit	Built
Contenedores	Started
Probabilidad	5.4075%
Clasificación	Riesgo bajo
Umbral operativo	75%

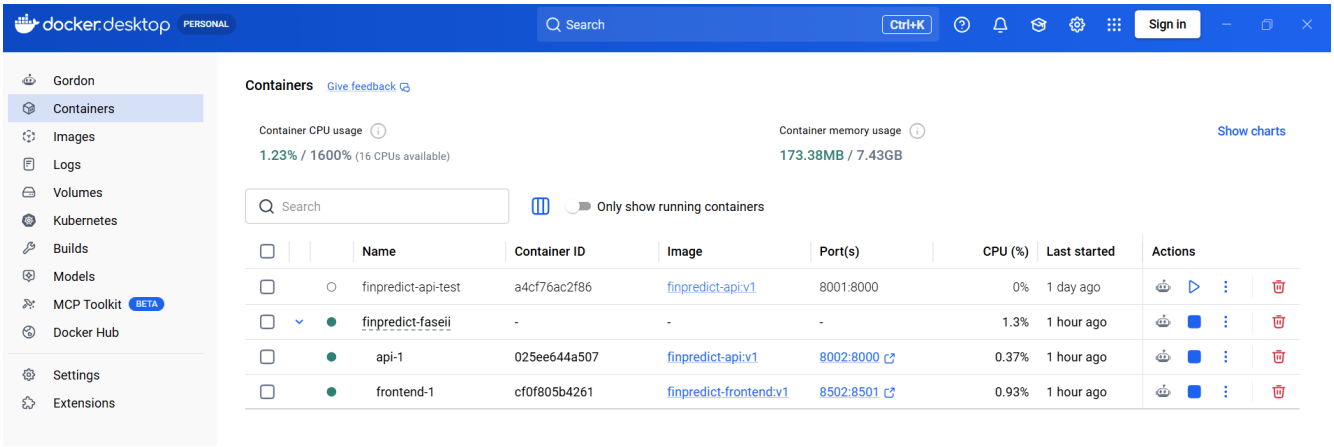
Conclusión: La reconstrucción mediante Docker Compose finalizó correctamente y la inferencia del caso de referencia coincidió con los resultados obtenidos previamente. La prueba utilizó caché de Docker y se realizó en el entorno de desarrollo existente. Por tanto, no constituye una validación de instalación desde cero en un equipo limpio.

7.9.1. Verificación visual de la arquitectura en Docker Desktop

Objetivo: Documentar la ejecución simultánea del frontend y backend bajo un mismo proyecto Docker Compose. **Resultado observado:**

Componente	Contenedor	Imagen	Puerto publicado	Estado
Backend	`api-1`	`finpredict-api:v1`	`8002:8000`	En ejecución
Frontend	`frontend-1`	`finpredict-frontend:v1`	`8502:8501`	En ejecución

Docker Desktop mostró ambos contenedores agrupados en el proyecto `finpredict-faseii`, con indicadores de ejecución activos. **Evidencia:** `docs/evidencias/24_docker_desktop_arquitectura.png` **Alcance:** Esta captura acredita la ejecución simultánea de los contenedores. La comunicación funcional y la inferencia del modelo se respaldan con las pruebas HTTP y end-to-end documentadas anteriormente.



Evidencia visual · 24_docker_desktop_arquitectura.png · Vista de la arquitectura en Docker Desktop.

7.10. Conclusión de las pruebas Docker

Las pruebas realizadas respaldan que FinPredict puede construir y reconstruir sus imágenes desde los Dockerfiles, iniciar servicios independientes, comunicarlos mediante Docker Compose y producir una inferencia consistente para el registro sintético de referencia. También se verificaron el reinicio de los servicios, su disponibilidad HTTP, el rechazo de tres entradas inválidas y la conservación del resultado predictivo después de reconstruir las imágenes. La reconstrucción aprovechó las capas de caché disponibles; por lo tanto, no equivale a una instalación desde cero en un entorno completamente limpio. No se realizaron pruebas de concurrencia, carga, alta disponibilidad, escalamiento horizontal, despliegue público o recuperación automática ante fallos de infraestructura. Estos aspectos quedan fuera del alcance de la validación actual.

8. Limitaciones, riesgos y alcance de los resultados

- **Datos sintéticos:** La información utilizada es simulada y no representa una cartera hipotecaria real ni permite inferir directamente desempeño en producción.
- **Desempeño predictivo:** La consistencia de resultados demuestra reproducibilidad de inferencia en los casos probados, no exactitud, calibración, equidad ni ausencia de sesgos en población real.
- **Casos extremos:** Los cinco escenarios seleccionados no cubren todas las combinaciones posibles de variables o condiciones financieras.
- **Reglas financieras:** Las restricciones de entrada son preliminares; requieren validación normativa, operativa y de negocio antes de una implementación institucional.
- **Seguridad:** No se implementó en estas pruebas un esquema empresarial de autenticación, autorización, cifrado TLS de extremo a extremo, gestión centralizada de secretos ni protección integral de datos.
- **Operación:** No se validaron métricas de rendimiento, monitoreo centralizado, alertamiento, tolerancia a fallas, balanceo de carga ni recuperación automática avanzada.
- **Portabilidad:** Se comprobó ejecución local con Docker Desktop y contenedores Linux. Aún corresponde validar una reconstrucción desde una copia limpia del repositorio para cerrar la reproducibilidad del paquete entregable.
- **Dependencias:** Se fijaron versiones explícitas de bibliotecas principales, pero no se verificó todavía un bloqueo completo de todas las dependencias transitivas.
- **Distribución:** El despliegue en la nube no ha sido ejecutado; su procedimiento propuesto está documentado en el manual técnico correspondiente.
- **Uso previsto:** FinPredict es un prototipo académico para priorización preventiva. **Sus resultados no deben utilizarse para aprobar, rechazar o modificar créditos reales.**

9. Estado consolidado de validación

9.1. Pruebas completadas

Actividad	Estado	Referencias
Consistencia histórica de 33,603 registros	Completada	Evidencia 01
Inferencia sobre cinco escenarios extremos	Completada	Evidencia 02
15 pruebas de validación de entradas	Completada	Evidencia 03
Disponibilidad, predicción y rechazos mediante API local	Completada	Evidencias 04–09
Cinco pruebas HTTP automatizadas	Completada	Evidencia 10
Integración Streamlit–FastAPI local	Completada	Evidencia 11
Desconexión y recuperación manual de API	Completada	Evidencias 12–13
Construcción y ejecución de API en Docker	Completada	Evidencias 14–16
Construcción de imagen Streamlit	Completada	Evidencia 17
Integración end-to-end mediante Docker Compose	Completada	Evidencias 18–19
Reinicio y estado de contenedores	Completada	Evidencia 20
Validaciones HTTP 422 dentro de Docker	Completada	Evidencias 21–23

9.2. Estado de cierre documental y próximos controles

Entregables documentales preparados:

- README general consolidado con arquitectura, requisitos, ejecución Docker Compose y enlaces al manual e informe.
- Manual técnico de despliegue y operación elaborado en Markdown y PDF; la implementación AWS se mantiene como propuesta no ejecutada.
- Informe técnico de validación y pruebas elaborado en Markdown y PDF, con 24 evidencias visuales integradas.
- Auditoría local de estructura y enlaces documentales completada: 20 archivos esenciales encontrados, 24 imágenes PNG reportadas y cero enlaces locales rotos en la revisión realizada.

Controles de entrega y reproducibilidad que permanecen sin acreditación:

- Confirmar una reconstrucción desde una copia limpia del repositorio en un entorno independiente y revisar las dependencias transitivas del paquete.
- Confirmar la publicación y el versionado final del repositorio GitHub de Fase II.
- Revisar el contenido del ZIP final y verificar que los dos PDF y las 24 capturas se incluyan cuando se prepare el paquete definitivo.

Las actividades documentales preparadas no equivalen por sí solas a publicación en GitHub o a validación de despliegue cloud.

10. Conclusión general

La Fase II de FinPredict logró integrar un modelo de regresión logística serializado con un backend FastAPI y una interfaz Streamlit; comprobar sus resultados respecto a una referencia histórica; validar entradas y solicitudes HTTP; y ejecutar el flujo completo mediante dos contenedores coordinados con Docker Compose. Las pruebas registradas muestran funcionamiento satisfactorio en el entorno local evaluado, incluyendo recuperación manual de la API, controles básicos de validación y consistencia de inferencia para el crédito sintético de referencia. Los controles pendientes se concentran en **la verificación desde una copia limpia del repositorio, la publicación en GitHub y la revisión del paquete de entrega**. Los resultados deben interpretarse dentro del alcance de un prototipo académico con datos sintéticos, sin equipararlos a una certificación de operación bancaria productiva.

Anexo A. Inventario de evidencias

Todas las capturas se almacenan en `docs/evidencias/`.

N.º	Archivo	Verificación
01	`01_consistencia_historica.png`	Consistencia del modelo exportado
02	`02_validacion_casos_extremos.png`	Cinco casos extremos
03	`03_validacion_entradas.png`	Quince pruebas de entrada
04	`04_api_health.png`	Salud de la API local
05	`05_api_primera_prediccion.png`	Primera inferencia HTTP local
06	`06_api_validacion_financiera.png`	Restricciones financieras en API local
07	`07_api_ingreso_negativo.png`	Rechazo de ingreso negativo local
08	`08_api_campo_obligatorio.png`	Rechazo de campo faltante local
09	`09_api_tipo_incorrecto.png`	Rechazo de tipo incompatible local
10	`10_api_pruebas_automatizadas.png`	Pruebas Pytest de API
11	`11_integracion_streamlit_fastapi.png`	Integración local de frontend y backend

N.º	Archivo	Verificación
12	12_frontend_api_desconectada.png	Error controlado con API desconectada
13	13_frontend_recuperacion_api.png	Recuperación de inferencia local
14	14_docker_build_api.png	Construcción de imagen API
15	15_docker_api_health.png	Salud de API en contenedor individual
16	16_docker_api_prediccion.png	Predicción desde Docker individual
17	17_docker_build_frontend.png	Construcción de imagen Streamlit
18	18_docker_compose_frontend.png	Interfaz Streamlit mediante Compose
19	19_docker_compose_prediccion.png	Predicción end-to-end mediante Compose
20	20_docker_compose_servicios_activos.png	Servicios Compose en estado Up
21	21_docker_validacion_ingreso_negativo.png	Restricción financiera en Compose
22	22_docker_campo_obligatorio.png	Campo faltante en Compose
23	23_docker_validacion_tipo_incorrecto.png	Tipo incompatible en Compose
24	24_docker_desktop_arquitectura.png	Ejecución simultánea de FastAPI y Streamlit en Docker Desktop